# Take-Home Exercise: Size-Level Availability Processor

**Suggested effort:** 3-4 hours
**Follow-up discussion:** 1 hour

A fashion marketplace receives seller stock changes for a specific product, size, and country. Build a small service that enriches each change with canonical product context, updates an Availability API, and publishes a normalized event for downstream consumers.

The scenario, APIs, models, and example data are hypothetical. They do not describe internal Farfetch systems.

## Start here

This document describes **what to build**. The accompanying README explains **how to run the provided mocks and send a sample input**.

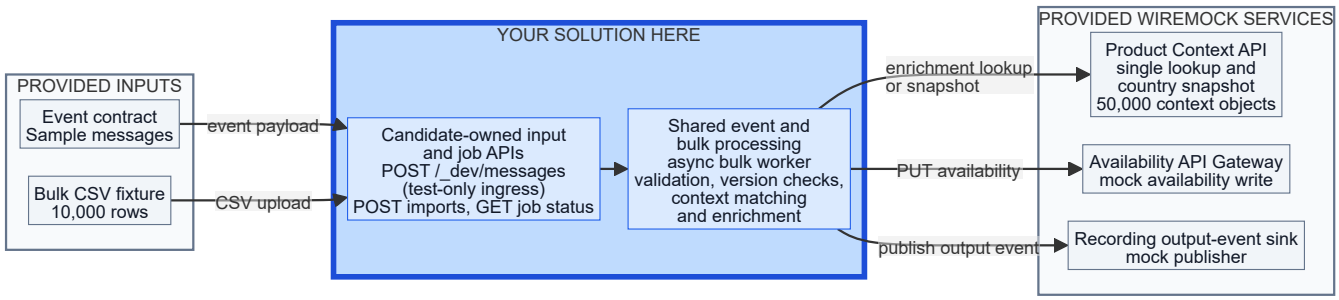
Start with one inventory event, then add a CSV import that reuses the same business flow. Read the ownership diagram first, start the provided mocks, and follow the numbered sections below.

Use any implementation language. You choose the internal architecture, persistence, and production broker. **You do not need to run a real broker, build a frontend, or deploy the service.** Describe your production broker choice and how you would connect it to your processing logic in your solution's README.

## 1. See what to implement

| Component | Who builds or supplies it? |
|---|---|
| `POST /_dev/messages` to receive a test event | **You implement this on your service.** |
| `POST /v1/availability/imports` and `GET /v1/availability/imports/{jobId}` | **You implement these on your service.** |
| Validation, version checks, enrichment, downstream calls, and asynchronous bulk processing | **You implement this logic.** |
| Sample event, 10,000-row CSV, and 50,000-object context snapshot | Included as ready-to-use JSON/CSV files. |
| Product Context API, Availability API Gateway, and output-event HTTP sink | Provided by WireMock at `http://localhost:8080`. |
| Real message broker | Not required locally; describe your production choice. |

The gray components are provided; the blue **YOUR SOLUTION HERE** box is your implementation. The boxes show responsibilities, not mandatory classes or separate services. The main flow is left to right; response arrows are omitted for readability.




If your viewer does not render the diagram, use this plain-text version:

```
PROVIDED INPUTS                  YOUR SOLUTION HERE
Sample event --curl/Postman----> POST /_dev/messages
Bulk CSV ---------------------> POST /v1/availability/imports
                                GET  /v1/availability/imports/{jobId}
                                  |
                                Shared processing + async bulk worker
                                  |
                                  +--> PROVIDED: Product Context API
                                  +--> PROVIDED: Availability API Gateway
                                  +--> PROVIDED: output-event HTTP sink
```

## 2. Start the provided mocks

You need a running Docker installation with Docker Compose. From the folder containing `compose.yaml`, run:

```
docker compose up -d
```

The sample event (`sample-event.json`), bulk CSV (`bulk-stock.csv`), and context snapshot (`product-contexts-PT.json`) are already included under `wiremock\__files`. No script or fixture-generation step is required. Open `http://localhost:8080/health` to check readiness.

**This starts only the provided dependencies, not your service.** Run your own application separately, on a different port. The examples below use `5000` for your application and `8080` for WireMock.

The supplied Compose file uses a pinned, prebuilt public `wiremock/wiremock` image with visible JSON mappings and response fixtures. No custom Dockerfile, launcher, Python, Java, or separate mock-server runtime is required on the host. Docker is required only for the mocks; no cloud account, production credentials, or external service is needed. There is no hidden evaluator. See the README for prerequisites, mock routes, failure triggers, and reset/stop commands.

## 3. Implement the single-event flow

For an inventory update that should be applied, your service:

1. Receives and validates the input event.
2. Uses the shared version rules in section 5 to avoid stale or duplicate updates.
3. Looks up canonical product context over HTTP.
4. Sends an availability update to the provided Availability API Gateway.
5. Publishes a transformed `SizeAvailabilityUpdated` event to the provided output sink.

## Receive an input event on your service

Implement `POST /_dev/messages` accepting this JSON contract. This is a test-only input route **you build**, not a route supplied by WireMock. It should call the same processing path that a real message-consumer adapter would use.

`countryCode` is a two-letter ISO 3166-1 alpha-2 code. Assume at-least-once delivery: messages may be repeated. Updates may also arrive out of order. `eventId` identifies a delivery and may change when the same business change is replayed.

```
{
  "eventId": "msg-0001",
  "eventType": "SellerInventoryChanged",
  "schemaVersion": 1,
  "occurredAt": "2026-10-20T08:00:00Z",
  "sellerId": "boutique-42",
  "productId": "style-123",
  "sizeId": "size-38",
  "countryCode": "PT",
  "availableQuantity": 3,
  "sourceVersion": 7
}
```

Once your service is implemented and running, send it the included sample event using `curl.exe`, Postman, or another HTTP client. For example, run this from the folder containing `compose.yaml` if your service listens on port `5000`:

```
curl.exe --request POST "http://localhost:5000/_dev/messages" `
  --header "Content-Type: application/json" `
  --data-binary "@wiremock\__files\sample-event.json"
```

Replace `5000` with the port your service uses. The `@` tells curl to read the request body from the included JSON file. This example uses PowerShell command-line syntax, not a supplied script; you can use Postman instead.

This request simulates a broker delivering an input event. No broker sends it automatically. Your service, not curl, performs the enrichment and downstream calls.

## Look up the product context

Call the provided Product Context API on WireMock:

```
GET /v1/products/{productId}/sizes/{sizeId}?countryCode={countryCode}
```

The API returns a `ProductContext` object with canonical product, size, and country fields. This is a simplified exercise model:

```
{
  "productId": "style-123",
  "productName": "Linen Shirt 123",
  "brand": {
    "id": "brand-42",
    "name": "Atelier Example"
  },
  "category": {
    "id": "cat-shirts",
    "name": "Shirts"
  },
  "color": {
    "id": "color-blue",
    "name": "Blue"
  },
  "productStatus": "ACTIVE",
  "sizeId": "size-38",
  "sizeLabel": "EU 38",
  "sizeSystem": "EU",
  "countryCode": "PT",
  "countryEnabled": true
}
```

The context lookup key is `(productId, sizeId, countryCode)`. Seller identity and available quantity come from the inventory event, not this API. Decide and document how to handle unknown products, inactive products, or products not enabled in the requested country.

## Write the availability update

Call the provided Availability API Gateway on WireMock:

```
PUT /v1/availability/{sellerId}/{productId}/{sizeId}?countryCode={countryCode}
```

The request body must include `availableQuantity` and `sourceVersion`. Include any canonical context fields you consider useful.

```
{
  "availableQuantity": 3,
  "sourceVersion": 7,
  "productContext": {
    "productName": "Linen Shirt 123",
    "brand": {
      "id": "brand-42",
      "name": "Atelier Example"
    },
    "sizeLabel": "EU 38",
    "sizeSystem": "EU"
  }
}
```

## Send the output event

Your service sends the transformed `SizeAvailabilityUpdated` JSON event to the provided WireMock sink using HTTP:

```
POST /mock-broker/v1/events
```

The full local URL is `http://localhost:8080/mock-broker/v1/events`. **This is an output-only HTTP mock, not a real message broker and not another endpoint you implement.** Do not send the inbound `SellerInventoryChanged` event to this route.

The output event must identify the seller, product, size, country, quantity, and source version, and include the canonical context used for enrichment. The sink records requests for inspection. The production broker and publisher adapter are your documented design choice; the local HTTP sink is only a test substitute.

```
{
  "eventId": "out-0001",
  "eventType": "SizeAvailabilityUpdated",
  "schemaVersion": 1,
  "occurredAt": "2026-10-20T08:01:00Z",
  "sellerId": "boutique-42",
  "productId": "style-123",
  "sizeId": "size-38",
  "countryCode": "PT",
  "availableQuantity": 3,
  "sourceVersion": 7,
  "productContext": {
    "productName": "Linen Shirt 123",
    "brand": {
      "id": "brand-42",
      "name": "Atelier Example"
    },
    "sizeLabel": "EU 38",
    "sizeSystem": "EU"
  }
}
```

### Check the result

Open `http://localhost:8080/__admin/requests` in Chrome to inspect the Product Context lookup, Availability API write, and output event your service sent. WireMock records these requests; it does not implement your business logic.

## 4. Add asynchronous bulk imports

Reuse the same validation, version checks, enrichment, availability write, and output-event flow for CSV rows. Bulk imports differ in how the input is accepted and how product context is fetched.

### Accept an upload and report progress

Implement `POST /v1/availability/imports` for a CSV file with these columns:

```
sellerId,productId,sizeId,countryCode,availableQuantity,sourceVersion
```

Reject malformed requests or missing headers synchronously. For an accepted upload, return `202 Accepted` and a `jobId`; do not wait for all rows to finish. Process rows asynchronously through the same business flow used for individual events. Provide:

```
GET /v1/availability/imports/{jobId}
```

The status response should include job state, total/processed/succeeded/failed row counts, and row-level errors. Define and document how partial failures are reported.

### Match rows to the context snapshot

For bulk processing, use the Product Context API country snapshot:

```
GET /v1/product-contexts/snapshot?countryCode={countryCode}
```

The response is a collection of `ProductContext` objects keyed by `(productId, sizeId, countryCode)`. Fetch one snapshot per distinct country in the file and match its contexts to the uploaded rows; do not make one single-item HTTP request per row. The snapshot order is unspecified. `sellerId` is part of the stock-version key, but not the context lookup key.

The package includes deterministic fixtures with 10,000 upload rows and 50,000 context objects for one country, ready to use without generating data yourself. No fixed, machine-specific timing threshold is used. In your README, explain the time and space complexity of your matching approach and what happens as both collections grow.

## 5. Apply the shared version and reliability rules

These rules apply to both input events and CSV rows.

### Stock identity and version scope

`sourceVersion` is monotonic only within this stock stream:

```
(sellerId, productId, sizeId, countryCode)
```

For example, version 8 supersedes version 7 for `(seller-A, style-123, size-38, PT)`. Version 8 for a different seller, product, size, or country belongs to an independent stream and must not be compared with it. A replay of the same business version may have a different `eventId`.

For this exercise, inventory events and CSV rows carry versions from the same authoritative sequence for each stock key. Version checks apply across both input paths; a bulk import is not an override of newer stock.

### Stale bulk rows

For each row, compare its `sourceVersion` with the latest successfully applied version for the same `(sellerId, productId, sizeId, countryCode)`, whether that version came from an event or another bulk row. If the row's version is lower, it is stale and must not overwrite the current stock. Do not send an availability write or publish a new output event for that stale row. This rule must also hold if a newer update is processed while the bulk job is running.

For example, if `(boutique-42, style-123, size-38, PT)` is already at version `8` with quantity `3`, a bulk row for that same key with version `7` and quantity `10` must leave the stock at version `8`, quantity `3`. A different seller, product, size, or country is an independent stock key.

A stale row alone must not cause the entire accepted upload to be rejected; continue processing the other rows. Document whether stale rows are reported as skipped or failed and how they are reflected in the job counts and row-level results.

### Dependency failures, retries, and flow control

The provided mocks have documented success and failure scenarios:

- Product Context lookup can return `404`, `429` with `Retry-After`, or `503`.
- The Availability API Gateway and recording event sink can experience transient failures.
- Messages may be duplicated, replayed with a new `eventId`, or delivered out of order.

Describe your retry and idempotency decisions, how you avoid applying stale versions, and what happens if the gateway update succeeds but event publication fails. Explain how you would control concurrency and avoid overwhelming dependencies during a burst or large upload. No load test is required.

## 6. Submit your solution

Submit your work via email or a public Git repository. Include:

- Runnable application source.
- Focused tests for the core flow and important failure cases.
- A README with run instructions, assumptions, design decisions, retry/idempotency behavior, partial-failure handling, and the production broker you would choose.
- A short explanation of the time and space complexity of bulk context matching.

Suggested effort is 3-4 hours. Production-grade completeness is not expected. In the one-hour follow-up, be prepared to walk through the design, explain tradeoffs, and discuss how you would adapt it if the load or failure conditions changed.